

Unity-MCP Sample Prompts

Related: [Define matrix](#) · [API coverage](#) · [MCP README](#)

Situation-based **sample prompts** for calling Unity MIDI Plugin MCP tools from MCP clients such as Cursor. These are category-level examples, not an exhaustive list of every Tool ID.

How to use

1. Add [Unity-MCP](#) (`com.ivanmurzak.unity.mcp`) to the project and configure Cursor under **Window** → **AI Game Developer**.
 2. Paste a prompt below into chat (rewrite paths, BPM, deviceId, etc. for your environment).
 3. Optional-dependency categories do not expose tools unless the matching `FEATURE_*` is present. See [define-matrix](#).
 4. Send, monitor, and most playback operations **require Play Mode**. In Edit Mode you get `[Error] ... Play Mode ...`.
-

1. Meta / diagnostics

When to use: First time with MCP, unsure which integrations are enabled, looking for docs, or safely adding a define.

Main tools: `midi-features-status` · `midi-features-enable` · `midi-integration-docs`

Check that Unity MIDI Plugin MCP is configured.

1. Summarize `FEATURE_*` / package / asm / VST detection via `midi-features-status`
2. Point me to Chunity and MPTK setup docs via `midi-integration-docs`

If adding a define, use `allowlist` only and `midi-features-enable` with `confirm=true`

I want to touch a VST3 host, but this repo's MIDI MCP is assumed not to have `vst3-*`. Confirm whether `jp.kshoji.unity.vst3nativehost` is present via `midi-features-status`, then guide me to the VST-side plan with `midi-integration-docs` `topic=vst3`.

2. Core (init / devices / send / monitor)

When to use: Quick check that real or virtual OUT receives data. Split TX/RX with Monitor. Convert note names and numbers.

Main tools: `midi-init` · `midi-devices-list` · `midi-send-note` / `cc` / `pc` · `midi-monitor-read` / `clear` · `midi-note-utility` · `midi-panic`

Enter Play Mode, then:

`midi-init` → `midi-devices-list` → send C4 (note=60, velocity=100) to the first OUT with confirm NoteOn via `midi-monitor-read`. Finish with `midi-panic`.

Send CC1 (Modulation) with channel=0, value=64 and check it appears in the monitor. Pick deviceId from an OUT in `midi-devices-list`.

Convert note name D#4 to a number, and number 72 to a note name, with `midi-note-utility`

3. SMF / UMP sequence & recording

When to use: Assetize `.mid` / `.midi2` and play with `SmfPlayer`. Inspect tempo maps. Record input into a Sequential Asset.

Main tools: `smf-preview-info` · `smf-import-asset` · `ump-import-asset` · `smf-player-control` · `midi-recorder-control` · `tempo-map-extract` · `ump-sequencer-control`, etc.

Summarize Assets/sample.mid with `smf-preview-info` (tracks, tempo, length).

If OK, create a `MidiSequenceAsset` with `smf-import-asset`.

Then create / assign `SmfPlayer` in the scene via `smf-player-control`,

Play Mode: Play → Stop after a few seconds.

Set up `MidiRecorder` in the current scene; in Play Mode start → keyboard input → stop save-asset to persist the recording.

List tempo change points from an imported Sequence with tempo-map-extract.

4. Gameplay (Router / Filter / Virtual / Settings)

When to use: Drop a NoteOn → UnityEvent scaffold quickly. Filter by channel/device. Inject into a virtual device to test a handler alone. Change Project Settings.

Main tools: midi-setup-router · midi-note-tracker-state · midi-filter-configure · midi-virtual-device · midi-virtual-device-inject · midi-project-settings-get / set

Create MidiInputMap + MidiInputRouter with midi-setup-router, add one NoteOn → UnityEvent binding. Briefly explain virtual-device inject for smoke

In Play Mode register midi-virtual-device, inject NoteOn C4, and check held notes increase via midi-note-tracker-state.

Summarize current MIDI Project Settings with midi-project-settings-get; if changes are needed, propose a diff then midi-project-settings-set (honor confirm w

5. Timeline / Animator / Input System / Visual Scripting

When to use: Wire SMF playback/record tracks on a Director. Drive Animator params from MIDI. Round-trip with Input System Synthetic Devices. Register VS nodes only (auto graph editing is out of scope).

Main tools: midi-timeline-* · midi-animator-* · midi-inputsystem-* · midi-vs-register-nodes / midi-vs-status

Required defines: Timeline / Input System / VS each need their FEATURE_*

With FEATURE_USE_TIMELINE, use midi-timeline-setup-playback to assemble PlayableDirector + SmfPlayer + Playback Track/Clip in the current scene. Bind an existing SequenceAsset with bind-clip.

Add Driver + Mapping to an Animator with `midi-animator-add-driver`,
add a CC1 → Float parameter binding with `midi-animator-add-binding`.
In Play Mode, explain how to read `midi-animator-read-params`.

With `FEATURE_INPUT_SYSTEM` run `midi-inputsystem-setup-bridge`,
then propose `.inputactions` paths via `midi-inputsystem-list-controls` and `suggest-bindi`

Check registration with `midi-vs-status`; if incomplete, `midi-vs-register-nodes`.
Graph editing can stay manual – report registration and missing deps only.

6. Scriptable Audio

When to use: Bootstrap a metronome or DSP Sequence on Unity 6 Scriptable Audio and verify playback/clock.

Main tools: `midi-sa-bootstrap-metronome` / `smf` / `ump` · `midi-sa-transport` · `midi-sa-clock-mode` · `midi-sa-validate`

Requires: `FEATURE_SCRIPTABLE_AUDIO` (Unity 6000.3+)

With `FEATURE_SCRIPTABLE_AUDIO`, bootstrap metronome (bpm=120) → fix via `midi-sa-validate`
then use `midi-sa-transport` in Play Mode until the click is audible.

Put an existing `MidiSequenceAsset` on the DSP path with `midi-sa-bootstrap-smf`,
and briefly demo `clock-mode` and `transport` (Play/Stop/Seek).

7. Chunity (Chuck)

When to use: No sound due to missing deps or Upstream PRs. Scaffold Bridge + PatchHost. Avoid poly double-trigger. Confirm Chuck Event → MIDI OUT in Monitor. Touch Generator / Timeline integration.

Main tools: `midi-chunity-diagnostics` / `validate-scene` / `setup-bridge` / `run-patch` / `set-global` · `poly-setup` · `event-to-midi` · `generator-*`, etc.

Requires: `FEATURE_CHUNITY` (Generator needs `FEATURE_CHUNITY_SCRIPTABLE_AUDIO`)

Resource example: `midi://chunity/scene-status`

Triage no sound:

Report Main/Sub/Bridge/PatchHost/define/Upstream PR/WebGL ArraySyncer notes via `midi-chunity-diagnostics` and `midi-chunity-validate-scene` (or Resource `midi://chunity/`
If PRs are missing, decide whether `apply-upstream-prs` (`confirm=true`) is needed.

Place Bridge + PatchHost with `setup-bridge`, `run-patch` a short `.ck` that receives noteC
In Play Mode send C4 and confirm `midiNote` / `noteOn` globals via `poll-global`.

Configure poly without double-trigger via `midi-chunity-poly-setup` (`useDefaultConventi`
and check `voiceId` / `activeNotes` under overlapping `NoteOns`.

Add an Event→MIDI binding with `midi-chunity-event-to-midi`,
fire the Event from the patch → confirm OUT with `midi-monitor-read`.

8. MPTK (Maestro)

When to use: Hear sound via `mptk:internal` . Try SMF Preview Audio (MPTK). Touch Pro Writer / Pipeline / External. Mirror to SA DSP.

Main tools: `midi-mptk-bootstrap` / `validate-setup` / `send-test-note` / `panic` · `preview-smf` · `global-settings` · `distance-audio` · Pro `effects-configure` / `voice-lifecycle` / `chord-progression` · `writer-*` / `pipeline-*` · `dsp-*-mirror` · experimental `velocity-attenuation`

Requires: `FEATURE_USE_MPTK` (Pro: `MPTK_PRO`)

Resource example: `midi://mptk/setup-report`

With `FEATURE_USE_MPTK`:

Play Mode → `midi-init` → `midi-mptk-bootstrap` → `validate-setup` →
`midi-mptk-send-test-note (C4)` for sound check → `midi-mptk-panic`.
On failure, summarize Resource `midi://mptk/setup-report`.

Play a `MidiSequenceAsset` (or `.mid`) with `midi-mptk-preview-smf` like Preview Audio (MPT then panic.

MPTK_PRO: `writer-load` → `writer-play` (or `write-temp`).
Optionally external-play file:// or https:// and watch Monitor via `InputAdapter`.

In Pro Pipeline, configure `ch9 Drop` + a simple inject rule with `pipeline-configure /` then read `midi://mptk/pipeline/stats` after playback. When using `InputAdapter`, prefer

`FEATURE_USE_MPTK`: `midi-mptk-global-settings get/set runInBackground`;
`midi-mptk-distance-audio` apply `DistanceAttenuation` (distinct from `Spatializer Track/C`
MPTK_PRO: `midi-mptk-effects-configure` apply `EnableFilter`;
`midi-mptk-voice-lifecycle` pause then resume;
`midi-mptk-chord-progression` list `genre=Pop` then play `uplifting_pop` (generation, not n

9. Networking

When to use: Want a UDP Hub/Client scaffold and RTT. Want only a Mirror / NGO / WSNet2 bridge foothold (vendor packages separate).

Main tools: `midi-net-hub-client-setup` · `midi-net-rtt` · `midi-net-discovery-status` · `midi-net-bridge-setup` · `midi-wsnet2-sync-defines`

Requires: `FEATURE_MIDI_NETWORK`

Resource example: `midi://network/status`

With `FEATURE_MIDI_NETWORK`, set up loopback Hub/Client via `midi-net-hub-client-setup` (After Play Mode + `midi-init`, take RTT with `midi-net-rtt ping=true`. Optionally also `discovery-status` and Resource `midi://network/status`.

After Hub setup, `midi-net-bridge-setup` with `framework=mirror` (or `netcode` / `wsnet2`). For `WSNet2`, run `midi-wsnet2-sync-defines` first. Do not commit vendor packages – repor

10. MIDI 2.0 / MPE / MIDI-CI

When to use: Enumerate/send UMP devices, check MPE zones, try Capability Inquiry (Discovery).

Main tools: `midi2-devices-list` · `midi2-send-ump` / structured `midi2-send-*` · `mpe-zone-status` · `midi-ci-discover`

In Play Mode: `midi2-devices-list (initialize=true)` → send a MIDI 2.0 Note On UMP (or `midi2-send-channel-voice`) to a valid OUT, and report the result.

Assuming a CI-capable device is connected on IN+OUT, run `midi-ci-discover` and summarize `discoveredCount` / `MUID` / `capability` flags.

Explain `mpe-zone-status` assuming zones are already set up (or `setupIfMissing`).

11. Transport / Foundation (helpers)

When to use: Session status for RTP / BLE / Nearby / UDP2; entry points for output routing, Clock, latency calibration. Full deep hardware UI recreation is out of scope.

Main tools: `midi-rtp-session` · `midi-ble-control` · `midi-nearby-control` · `midi2-udp-session` · `midi-transport-status` · `midi-output-routing` · `midi-clock-sync` / `output` · `midi-latency-calibration` · `midi-device-selection`, etc.

List available transports and caveats with midi-transport-status.
For RTP, show only the minimal midi-rtp-session steps (deep Companion setup is out of

Inspect midi-output-routing and briefly show how to steer to a specific deviceId.
If useful, one sentence on midi-clock-sync vs midi-clock-output.

Cheat sheet (situation → section)

Situation	See
Is MCP working / which FEATURE is on?	§1 Meta
Does C4 reach a real device?	§2 Core
Place SMF, play & record	§3 SMF
Game-side input wiring	§4 Gameplay
Timeline / Animator / Input System / VS	§5
Scriptable Audio click audible?	§6
ChuckK integration diagnose → sound	§7
Maestro sound / Preview / Pro	§8
Network sync foothold	§9
UMP / MPE / CI	§10
Entry for RTP / BLE / Clock	§11

VST host operations (`vst3-*`) are out of scope for this document. See [Unity-VST3-Bridge](#) docs instead.